

BUN

Entorno de ejecución de JavaScript y TypeScript

Manual

Autora: Pérez Repiso, Carmen

Curso: 2.º DAW ·

Módulo: OPT

Fecha: 13 de octubre de 2026

BUN 1
2. ¿Qué es y para qué sirve? 2
a) Qué es y para qué sirve 2
b) Qué problema resuelve y por qué surgió 2
c) Quién la crea o mantiene y versión actual 2
3. Conceptos clave 2
4. Requisitos e instalación 5
a) Requisitos previos 5
b) Comandos de instalación paso a paso 5
c) Estructura de carpetas generada 5
d) Capturas de pantalla que demuestran que funciona 7
5. Ejemplo básico de uso 9
a) Mini Proyecto propio: recorrer una lista de tecnologías 9
b) Código comentado, explicado por partes 9
c) Resultado final con capturas de pantalla 11
d) Efecto de la tecnología en el ejemplo 11
6. Ventajas, inconvenientes y conclusión 11
Ventajas 11
Inconvenientes 13
Conclusión y valoración personal 13
7. Recursos y bibliografía 13

2. ¿Qué es y para qué sirve?

a) Qué es y para qué sirve

Bun es una herramienta para programar. Sirve para ejecutar programas escritos en JavaScript y TypeScript, dos lenguajes muy usados en la web, directamente en tu ordenador y no solo dentro del navegador. Por ejemplo, para crear un servidor: un programa que se queda esperando y responde cuando alguien le pide algo, como cuando tu navegador pide una página. Además, trae incluidas una herramienta para instalar librerías (código de otras personas que puedes reutilizar) y otra para hacer tests (pruebas automáticas), todo en un solo programa.

b) Qué problema resuelve y por qué surgió

Hasta hace poco había que juntar varias herramientas: Node.js para ejecutar el código, npm para las librerías, Jest para los tests y otra para empaquetar. Eran muchas piezas que instalar y configurar, y algunas iban lentas. Bun surgió para reunirlo todo en una sola herramienta, rápida y fácil de configurar, y en gran parte compatible con Node.js para poder seguir usando sus librerías.

c) Quién la crea o mantiene y versión actual

Lo creó Jarred Sumner y salió en 2022; la versión 1.0 llegó en septiembre de 2023. Al principio lo desarrollaba su empresa, Oven, y en diciembre de 2025 pasó a formar parte de Anthropic. Sigue siendo de código abierto (licencia MIT) y su código está en GitHub (oven-sh/bun). He usado la versión 1.4.3.

3. Conceptos clave

Antes de empezar, unas palabras que van a salir todo el rato:

- Terminal: una ventana donde, en vez de hacer clic, escribes órdenes para el ordenador.
- Comando: cada una de esas órdenes, por ejemplo bun - - version .
- JavaScript y TypeScript: lenguajes de programación. TypeScript es JavaScript con unas «etiquetas» extra (los tipos) que ayudan a pillar errores antes de ejecutar.
- Servidor: un programa que espera peticiones y las contesta.
- Librería: código hecho por otras personas que puedes usar en tu proyecto.
- Test: un programa pequeño que comprueba que otra parte del código hace lo que tiene que hacer.

Y ahora, las cinco ideas que hay que entender antes de usar Bun.

I. Runtime (el que ejecuta tu código)

Un programa es solo un archivo de texto. Hace falta algo que lo lea y haga lo que pone, y a eso se le llama runtime. Bun es un runtime: para ejecutar un archivo escribes bun run y su nombre. Por ejemplo, este archivo muestra un mensaje:

```
//hola.ts
console.log('HoladesdeBun');
```

y

para ejecutarlo escribes esto en la terminal y sale «Hola desde Bun»:

```
bunrunhola.ts
```

2. **TypeScript sin preparar nada**

Bun entiende los archivos.ts (TypeScript) sin que tengas que instalar ni configurar nada antes, cosa que con otras herramientas sí hace falta. En el ejemplo, number significa que a y b son números. Si lo ejecutas con bun run suma.ts, sale 5.

```
//suma.ts
functionsumar(a:number,b:nu
  mber):number{returna+b;
}
console.log(sumar(2,3));
```

3. **Gestor de paquetes (instalar librerías)**

Casi nadie hace un proyecto desde cero: se usan librerías de otras personas. El gestor de paquetes se encarga de descargarlas e instalarlas. Las que usa tu proyecto se apuntan en un archivo llamado package.json , que es como una lista de la compra. Las librerías descargadas se guardan en la carpeta node_modules .

```
bun add express     #añadeunalibrería(aquí,express)yla apunta en package.js
onbuninstall  #descargatodaslaslibrerías que aparecen en package.json
```

4. **Bun.serve (crear un servidor)**

Es la función de Bun para crear un servidor web. Le das una función que se ejecuta cada vez que llega una petición (por ejemplo, cuando alguien abre la página) y que decide qué contestar. Este servidor responde siempre «Hola desde mi servidor»:

```
Bun.se
  rve({
  port:
  3000,
  fetch(
  ){
    returnnewResponse('Holadesdemiservidor');
},
})
 ;
```

5. **Test runner (ejecutar tests)**

Un test comprueba que algo funciona. b u n t e s t busca los archivos que acaban en . test.ts y ejecuta las pruebas que llevan dentro. Este comprueba que 2 + 2 es 4:

```
import{test,expect}from'bun:test';

test('2+2son4',()=
  >{expect(2+2).to
  Be(4);
});
```

```
                                                          (4herramientas)
Antes:    Node.js   +    npm  +    Jest  +    empaquetador
ConBun:bunrun   +    buninstall  +    buntest    +    bunbuild   (1herramienta)
```

Esquema: de varias herramientas a una sola.

4. Requisitos e instalación

a) Requisitos previos

- Un ordenador con Windows, macOS o Linux y conexión a internet para descargar Bun.
- Una terminal. En Windows: pulsa Inicio, escribe PowerShell y dale a Intro. En macOS: búscala con la lupa de arriba a la derecha escribiendo Terminal. En Linux, normalmente con Ctrl + Alt + T.
- Un editor de código, que es como un bloc de notas pero para programar. Yo he usado Visual Studio Code, que es gratis: code.visualstudio.com.
- No hace falta tener Node.js instalado ni crearse ninguna cuenta.

b) Comandos de instalación paso a paso

Paso 1. Abre la terminal y copia el comando que corresponda a tu sistema. Pégalo con clic derecho o con Ctrl + V, pulsa Intro y espera a que termine.
En Windows (PowerShell):

```
powershell-c"irmbun.sh/install.ps1|iex"
```

En macOS o Linux:

```
curl-fsSLhttps://bun.sh/install|bash
```

Al terminar sale un mensaje en verde diciendo que Bun se ha instalado (captura 1). También te pide reiniciar la terminal, así que ciérrala y vuelve a abrirla.
Paso 2. Comprueba que Bun funciona escribiendo este comando. Debería salir un número de versión; en mi caso, 1.4.3 (captura 2).

```
bun--version
```

Paso 3. Crea el proyecto. Primero entra con c d en la carpeta donde quieras guardarlo (la mía es C:\web\sitios\Manual_Bun ; cambia la ruta por la tuya). Después mkdir mini-bun crea una carpeta llamada mini-bun, cd mini-bun entra en ella y bun init prepara dentro la estructura de un proyecto.

```
cd C:\ web\sitios\ Manual _ Bun

mkdir mini-bun
cd mini-bun
bun init
```

bun init te pregunta qué plantilla quieres (captura 3). Yo elegí Blank, que es la de un proyecto en blanco, y pulsé Intro. Bun crea entonces los archivos y se pone a descargar dos paquetes por su cuenta; en mi caso, 7 paquetes en menos de cuatro segundos (captura 4).

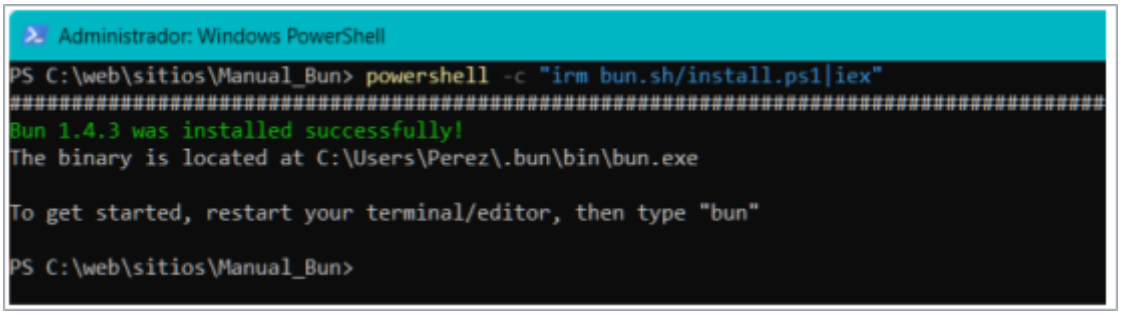
c) Estructura de carpetas generada

Al terminar, dentro de m i n i - b u n aparecen estos archivos y carpetas (captura 5):

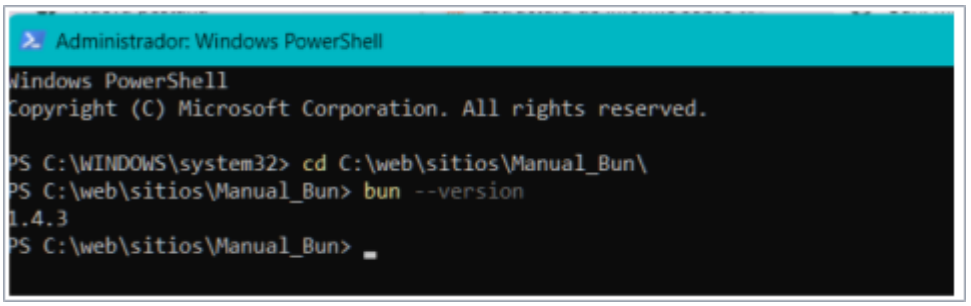
- **index.ts:** el archivo principal, por donde empieza a ejecutarse el programa. Bun lo crea con un mensaje de prueba, console.log (" Hello via Bun! ") ; (captura 6).
- **package.json:** describe el proyecto (nombre, versión y librerías que usa).
- **tsconfig.json:** la configuración de TypeScript. Sirve para comprobar el código con bun checky para el autocompletado del editor. De momento no hace falta tocarla.
- **README.md:** un texto para explicar de qué va el proyecto.

- **.gitignore:** lista de archivos que Git (el programa para guardar versiones de un proyecto) no tiene que subir.
- **bun.lock:** guarda las versiones exactas de las librerías instaladas.
- **node_modules/:** la carpeta donde se guardan los paquetes descargados. Aquí están @types/bun (ayuda al editor a conocer las funciones de Bun) y typescript .

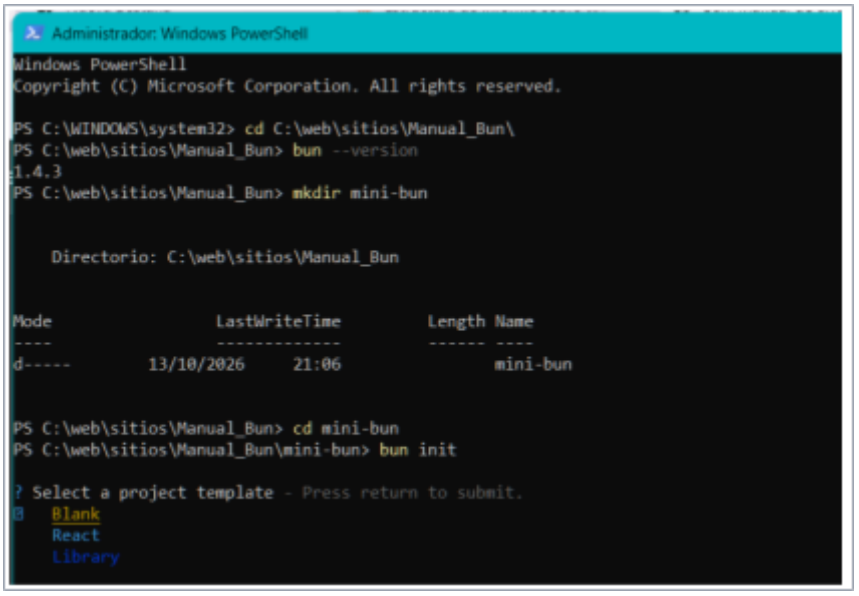
d) Capturas de pantalla que demuestran que funciona



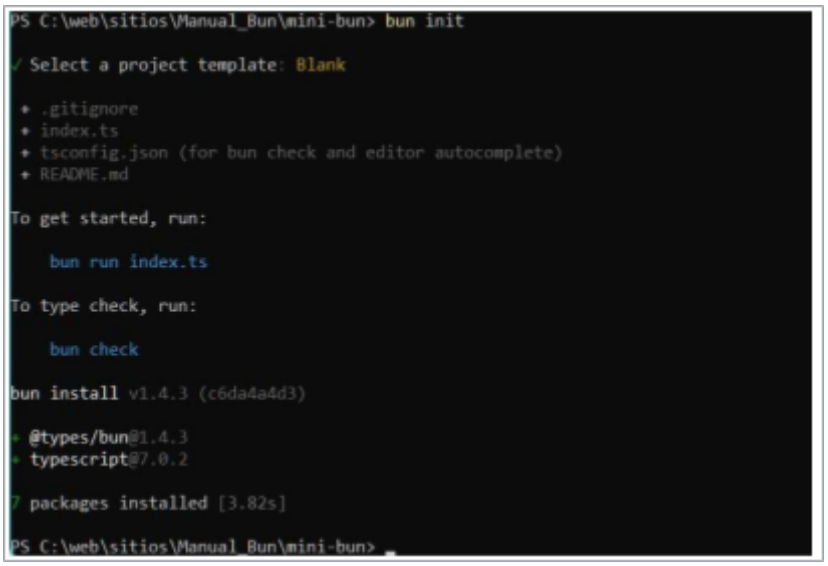
Captura 1. Instalación de Bun en PowerShell: el mensaje en verde confirma que se ha instalado la versión 1.4.3.



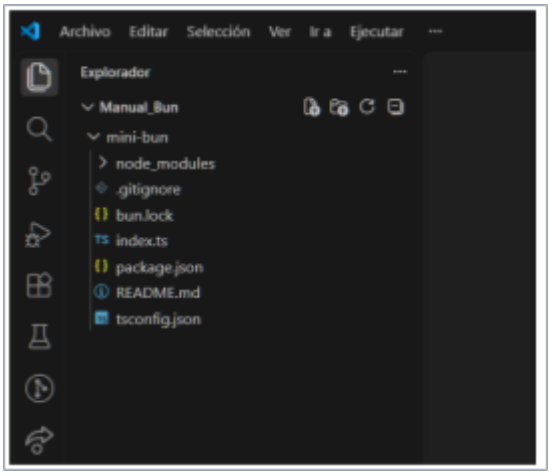
Captura 2. bun - -version devuelve 1.4.3, así que Bun funciona.



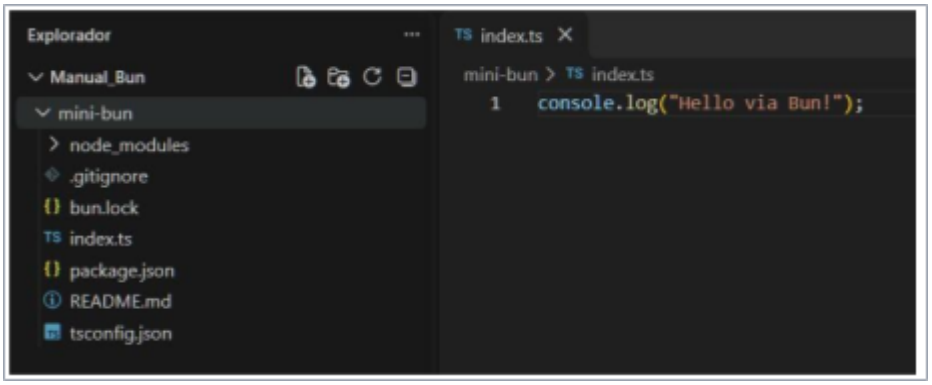
Captura 3. Creación de la carpeta mini-bun arranque de bun init, que pregunta por la plantilla.



Captura 4. Resultado de bun init con la plantilla Blank: archivos creados y 7 paquetes instalados.



Captura 5. Carpeta mini-bun abierta en Visual Studio Code, con los archivos que ha generado Bun (recorte del explorador de archivos).



Captura 6. El archivo index.ts que crea Bun, con su mensaje de prueba.

5. Ejemplo básico de uso

a) Mini Proyecto propio: recorrer una lista de tecnologías

El miniproyecto es un programa muy sencillo: tiene una lista con tres tecnologías (Node.js, Deno y Bun) y escribe un mensaje por cada una en la terminal. Es sencillo a propósito. Con él se ve que Bun ejecuta TypeScript directamente y se practican cosas básicas de cualquier programa: guardar datos en una lista, recorrerla y mostrar mensajes. Está todo en un archivo llamado tarea.ts, dentro de la carpeta mini-bun , y también lo puedes encontrar en la carpeta /ejemplo del repositorio.

b) Código comentado, explicado por partes

En Visual Studio Code, con la carpeta m i n i - b u n abierta, usa el botón «Nuevo archivo» del panel de la izquierda y llámalo exactamente
tarea.ts . Pega este código y guarda con Ctrl + S (captura 7):

```
// Un array de tecnologías para recorrer
const tecnologias: string[] = ["Node.js", "Deno", "Bun"];

console.log("--- PROBANDO LA VELOCIDAD DE BUN ---")

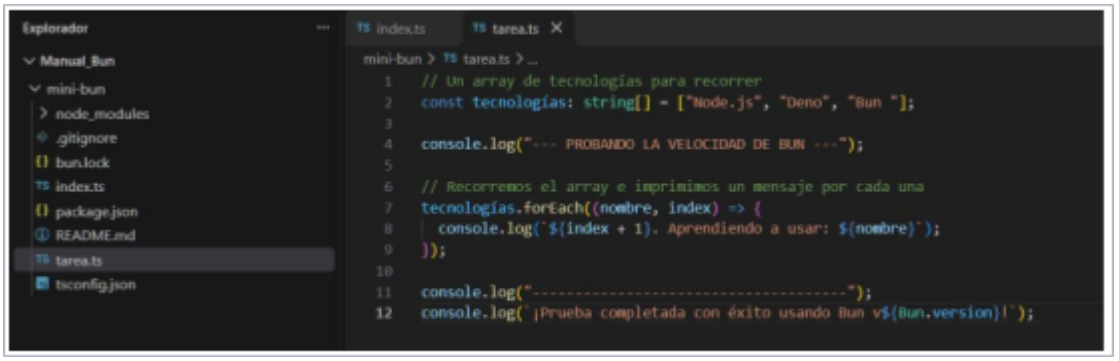
;

// Recorremos el array e imprimimos un mensaje por cada tecnología.
tecnologias.forEach((nombre, index) => {
  console.log(`${index + 1}. Aprendiendo a usar: ${nombre}`);
});

console.log(" ");
console.log(`¡Prueba completada en éxitos usando Bun v${Bun.version}!`);
```

Qué hace cada parte:

- const tecnologías: string [ ] = [. . .] : crea una lista (en programación se llama array) con tres textos. const significa que la variable no se va a cambiar y string [ ] le dice a TypeScript que es una lista de textos.
- console.log (. . .) : escribe un mensaje en la terminal. Lo uso para el título y para la línea de guiones.
- forEach : significa «para cada elemento de la lista, haz esto». En cada vuelta, n o m b r e es el elemento (Node.js, Deno, Bun) e index es su posición, que empieza en 0. Por eso se escribe i n d e x + 1 , para que la numeración empiece en 1.
- Las comillas del revés (` . . . `) permiten meter valores dentro de un texto con $ { . . . } . Así el mensaje cambia según el elemento.
- Bun.versión : es algo que solo existe cuando el programa se ejecuta con Bun, y devuelve el número de versión. Con él, el programa dice qué versión de Bun lo está ejecutando.



Captura 7. El archivo tarea.ts en Visual Studio Code, dentro de la carpeta mini-bun .

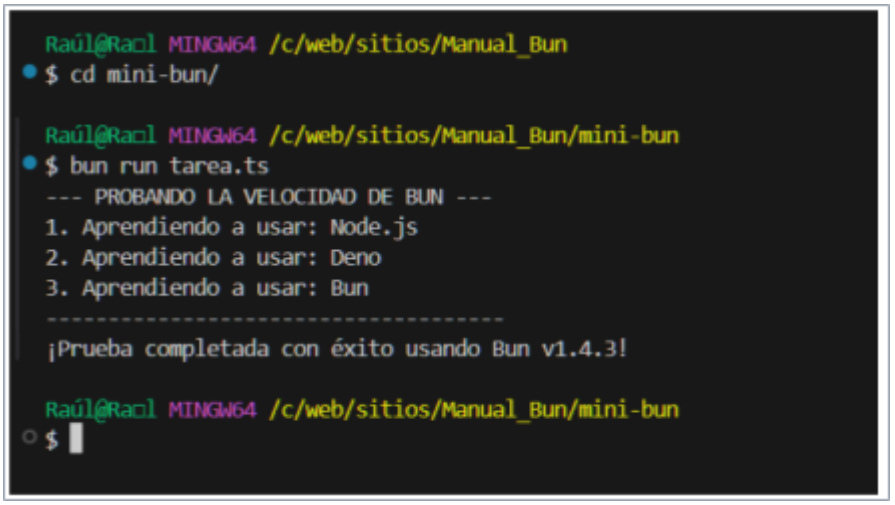
c) Resultado final con capturas de pantalla

En la terminal, dentro de la carpeta m i n i - b u n , ejecuta el archivo con un solo comando. No hay que compilar nada antes:

```
bun run tarea.ts
```

Debe salir esto (captura 8):

```
- - - PROBANDO L A VELOCIDAD DE BUN - - -
·  Aprendiendo usar : Node.js
·  Aprendiendo a usar : Deno
·  Aprendiendo a usar : Bun
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
¡ Prueba completada con éxito usando Bun v 1.4.3 !
```



Captura 8. Ejecución de bun run tarea.ts en la terminal de Visual Studio Code, con el resultado final.

Para comprobar que funciona de verdad, cambia un nombre de la lista (por ejemplo, pon «Python» en lugar de «Deno»), guarda y vuelve a ejecutar el comando: el mensaje cambia.

d) Efecto de la tecnología en el ejemplo

Bun no tiene ventanas ni botones, así que su efecto se ve en la terminal. Con un solo comando, bun run tarea.ts, ha ejecutado directamente un archivo TypeScript, sin compilar ni configurar nada, y ha mostrado el resultado, incluida su propia versión (captura 8). Además, el proyecto se creó con un par de comandos y quedó listo para usar (capturas 3 a 5).

6. Ventajas, inconvenientes y conclusión

Ventajas

- Lo tiene todo en uno: ejecuta programas, instala librerías y hace tests.
- Se instala en un momento y casi no hay que configurar nada. En mi caso, bun init preparó el proyecto y descargó 7 paquetes en menos de 4 segundos.
- Ejecuta TypeScript directamente.
- Arranca e instala librerías muy rápido.
- Es compatible en buena parte con Node.js, que es la herramienta más usada hasta ahora, y con sus librerías

Inconvenientes

- Es más joven que Node.js, así que tiene menos años de uso en proyectos grandes y menos respuestas en foros cuando algo falla.
- No es compatible al 100 % con Node.js, y alguna librería puede dar problemas.
- Muchas empresas ya trabajan con Node.js, así que cambiar no siempre compensa.

Conclusión y valoración personal

Para empezar me parece una herramienta muy cómoda: con un solo comando ya tienes un proyecto funcionando. Yo la usaría en proyectos pequeños o personales, como una aplicación sencilla o un prototipo, donde se agradece que sea rápida y que no haya que configurar casi nada. Para un proyecto grande de una empresa, antes de decidirme comprobaría que todas las librerías que necesito funcionan bien con Bun.

7. Recursos y bibliografía

- Web oficial de Bun: https://bun.sh
- Documentación oficial: https://bun.sh/docs
- Guía de instalación: https://bun.sh/docs/installation
- Servidor HTTP con Bun.serve: https://bun.sh/docs/api/http
- Test runner (bun test): https://bun.sh/docs/cli/test
- Bun is joining Anthropic (blog de Bun): https://bun.sh/blog/bun-joins-anthropic

Repositorio del manual y del ejemplo:
https://github.com/Carmen-pr/Manual-Bun-PerezRepiso_Carmen